
GENERATOR MELODII MIDI Z WYKORZYSTANIEM SIECI NEURONOWYCH

Daniel Augusto, Patryk Tomkowicz, Jakub
Warakomski

CELE PROJEKTU

- Celem projektu było stworzenie modelu LSTM (Long Short-Term Memory) oraz wytrenowanie go na zestawie danych: plikach MIDI zawierających tylko jeden instrument (pianino). Następnie zaimplementowano tworzenie plików MIDI reprezentujących sekwencję dźwięków wygenerowanych przez model zgodnie z kryteriami zadanymi przez użytkownika: tempo, tonacja, metrum oraz długość utworu (wyrażona w taktach).



SIEĆ LSTM

- Wybór implementacji naszego modelu z wykorzystaniem sieci neuronowych typu LSTM był spowodowany tym, że tego rodzaju sieć jest dobrze dostosowana do analizy i przetwarzania danych sekwencyjnych - którymi w tym przypadku są dane z plików MIDI. Kontekst muzyczny melodii oznacza, że nuty nie mogą być analizowane jako niezależne. Każda kolejna nuta jest zależna od wcześniejszej sekwencji. LSTM dzięki mechanizmom pamięci i bramek wejścia, wyjścia i zapominania potrafi uczyć się zależności pomiędzy zdarzeniami muzycznymi, a także decydować, które informacje z poprzedniego stanu są najbardziej istotne.
-

BAZA DANYCH

- Przygotowano bazę danych wejściowych, składającą się z 300 ponumerowanych plików MIDI, reprezentujących melodie zagrane na pojedynczym instrumencie: pianinie.

WCZYTANIE DANYCH

- Aby rozpocząć przetwarzanie danych, najpierw należało zaimplementować algorytm parsowania wczytanego pliku MIDI. Ponieważ ścieżki MIDI mogą zawierać metadane, szukano w pliku ścieżek zawierających wiadomości `note_on` z polem `velocity > 0`. Kiedy wyznaczona zostanie ścieżka zawierająca melodię, algorytm przechodzi do analizowania sekwencji wiadomości. Skorzystano tutaj z biblioteki `music21`, która parsuje pauzy i dźwięki. Ponieważ, dla poprawy jakości generowania melodii, zmodyfikowano wszystkie utwory do postaci monofonicznej, po wykryciu akordu algorytm wybiera najwyższy dźwięk.



OZNACZENIA NUT

- W przypadku wysokości dźwięku, z wiadomości MIDI wystarczy pobrać wartość pola pitch, która mapuje wysokości tonów od C w oktawie "-2" do F#/Fis w oktawie 8. W przypadku długości dźwięków, należy dokonać kwantyzacji do standardowych długości wykorzystywanych w notacji muzycznej. Długość każdej wiadomości note_on jest przypisywana do najbliższej zdefiniowanej długości dźwięku.

```
class Duration(Enum):
```

```
    WHOLE = (0, 4.0)
```

```
    HALF = (1, 2.0)
```

```
    QUARTER = (2, 1.0)
```

```
    EIGHTH = (3, 0.5)
```

```
    SIXTEENTH = (4, 0.25)
```

```
    HALF_DOTTED = (5, 3.0)
```

```
    QUARTER_DOTTED = (6, 1.5)
```

```
    EIGHTH_DOTTED = (7, 0.75)
```

```
    SIXTEENTH_DOTTED = (8, 0.375)
```

```
    HALF_TRIPLET = (9, 4/3)
```

```
    QUARTER_TRIPLET = (10, 2/3)
```

```
    EIGHTH_TRIPLET = (11, 1/3)
```

```
    SIXTEENTH_TRIPLET = (12, 1/6)
```

GENEROWANIE TOKENÓW

- Podczas przetwarzania pliku, generowane są tokeny. Pierwszy z reprezentuje dźwięki w formacie wysokość_długość. Drugi reprezentuje pauzy w formacie pauza_długość. Trzeci reprezentuje koniec utworu - jest to token <SONG_END>. Kiedy cały utwór został przetworzony, na wyjściu otrzymywana jest lista event_tokens, przechowująca wszystkie uzyskane tokeny.



PRZYGOTOWANIE DANYCH

- Budowany jest zbiór tokenów `all_events`, przechowujące wszystkie unikalne tokeny uzyskane podczas przetwarzania wejściowych danych. Dwa pomocnicze słowniki `event_to_int` oraz `int_to_event` mapują tokeny ze zbioru na numery i odwrotnie. Lista `encoded` zawiera tokeny reprezentowane jako liczby całkowite. Wartość `SEQ_LEN = 64` oznacza, że model widzi ostatnie 64 tokeny i dokonuje predykcji tokenu nr 65. Dane są przygotowane w postaci `X, y` - gdzie `X` oznacza sekwencję 64 tokenów a `y` oznacza faktyczną wartość tokenu następującego po tej sekwencji. Etykieta `y` jest kodowana metodą one-hot (na podstawie długości słownika), ponieważ wektor zwracany przez ostatnią warstwę modelu ma taką samą długość jak wygenerowany słownik tokenów.

CHARAKTERYSTYKA MODELU

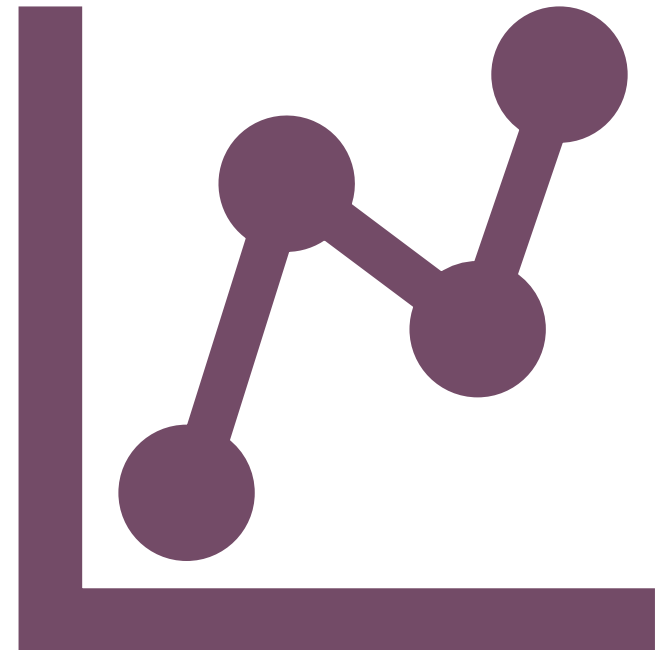
- Model składa się z trzech warstw. Warstwa Embedding reprezentuje każdy otrzymany token jako wektor 64 wartości liczbowych. Na początku liczby te są losowe, a podczas treningu aktualizowane są razem z wagami LSTM, na podstawie tego którego tokeny są podobne, występują razem albo pełnią podobną funkcję w sekwencji. Warstwa sieci LSTM składa się ze 128 neuronów pamięci opisujących rytm, powtarzające się frazy i występujące dźwięki. Zastosowano 20% dropout w celu lepszej generalizacji oraz 20% rekurencyjny dropout, po to, aby model nauczył się zasad muzycznych zamiast samych melodii z zestawu uczącego. Warstwa zamienia 128-liczbowy wektor na 904 liczby, po jednej dla każdego tokenu. Za pomocą funkcji aktywacji softmax wyniki zamieniane są na rozkład prawdopodobieństwa. Funkcją straty jest kategoriowa entropia krzyżowa, ponieważ jest to problem klasyfikacji wieloklasowej.
-

UCZENIE MODELU

- Podczas trenowania modelu stosowana jest metoda EarlyStopping, zapobiegająca przeuczeniu. Jeżeli błąd na zbiorze walidacyjnym (stanowiącym 20% zbioru uczącego) po trzech epokach nie poprawia się, to proces uczenia jest kończony. Maksymalna ilość epok wynosi 50. Przywracany jest zestaw najlepszych dotychczas uzyskanych wag. Dokładność uzyskana podczas trenowania naszego modelu wyniosła niemal 13%. Losowy klasyfikator przy 904 klasach uzyskiwałby teoretyczną dokładność równą ok. 0.11%.
-

PREDYKCJA

- Po treningu modelu generowane są pojedyncze tokeny, każdy kolejny wykorzystywany do przewidywania następnego elementu sekwencji. Na wejściu otrzymywany jest wektor prawdopodobieństw tokenów. Zastosowano hiperparametr temperatury podczas próbkowania (Temperature Sampling), który wyznacza czy rozkład należy spłaszczyć (> 1), nie zmieniać ($= 1$), lub uwypuklić różnice jeszcze bardziej (< 1). W przypadku naszego modelu ustawiono temperature = 0.9, dzięki czemu model jest bardziej zachowawczy podczas generowania melodii.



KARANIE PAUZ

- Podczas początkowych testów, zauważono, że wygenerowane melodie zawierają zbyt dużo pauz, przez co brzmią one nienaturalnie. Ponieważ token "REST" występuje zdecydowanie częściej niż poszczególne wysokości tonów w zbiorze testowym, zastosowano funkcję kary, aby obniżyć liczbę tokenów "REST" w wygenerowanych melodiach. Wszystkie warianty tokenów typu REST otrzymały 50% wyznaczonego prawdopodobieństwa. Dodatkowo, wprowadzono karę za pauzy występujące po sobie. Jeżeli poprzedni token był pauzą, to prawdopodobieństwo wszystkich tokenów reprezentujących pauzy spada do 20% oryginalnej wartości. Następnie dokonywana jest ponowna normalizacja prawdopodobieństw.



GENEROWANIE SEKWENCJI

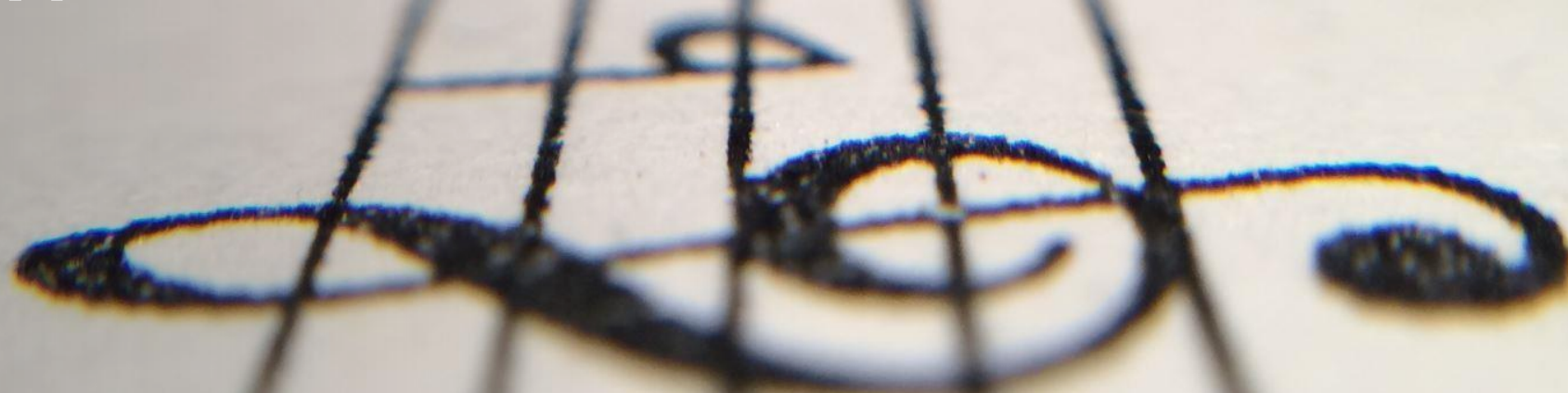
- W ramach generowania kolejnych elementów sekwencji, następny token losowany jest 10 najbardziej prawdopodobnych tokenów (top-k sampling). Dzięki losowemu wyborowi, model nie wybiera zawsze najlepszego tokenu, co sprawia, że melodie są mniej powtarzalne i przewidywalne.

PARAMETRY UTWORU



- Generowane melodie mogą zostać skonfigurowane w następujący sposób:
- Tempo
- Tonacja (transpozycja o określoną liczbę półtonów, tryb pozostaje domyślny)
- Długość utworu (w taktach)
- Metrum

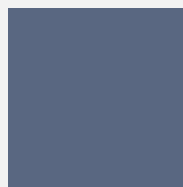
PRZYKŁAD - WYGENEROWANE UTWORY



OBSERWACJE I WNIOSKI



Zaimplementowany system pozwala na generację prostych, monofonicznych melodii bez znacznego dysonansu i kompletnego rytmicznego chaosu.



Można zaobserwować pewne podobne i powtarzające się motywy i frazy w różnych utworach.



Tempa spoza zakresu 50-150 BPM powodowały generację nienaturalnie brzmiących melodii.

PROPOZYCJE USPRAWNIENIA PROJEKTU



Zwiększenie zbioru
danych treningowych



Uczenie modelu
konkretnych wydarzeń
muzycznych



Bardziej zaawansowane
metody konfiguracji
tonacji



Analiza metrum



Polifonia, analiza
harmoniczna, akcenty,
itd.

KONIEC

Dziękujemy za uwagę!
